

CS1200 / CSCI E-120 Lecture 4

Reductions and static data structures

September 16, 2026

Announcements and reading

- Anurag's office hours: today 2:15–3:15, SEC 3.323; Friday 11 am–12 pm, Zoom.
- Please ask questions and give textbook feedback in *Perusall*.
- Pset 0 regrade request due tonight (11:59 pm)
- Hesterberg–Vadhan, Chapter 4.

Pset0 In-class reflection

- Before in-class reflection: R (114), R- (10), L (4), N (19)

Pset0 In-class reflection

- Before in-class reflection: R (114), R- (10), L (4), N (19)
- After in-class reflection: R(87), R- (7), L (33), N (15)

Pset0 In-class reflection

- Before in-class reflection: R (114), R- (10), L (4), N (19)
- After in-class reflection: R(87), R- (7), L (33), N (15)
- Our decision: not to be implemented for pset 0. Strictly enforced from pset 1 onwards.
- Understanding, owning and internalizing the solution you submit is crucial.

Easy vs hard problems

- We saw in previous lectures that SORTING can be solved fairly quickly - in $O(n \log n)$ time. We can call SORTING an *easy* problem.

Easy vs hard problems

- We saw in previous lectures that SORTING can be solved fairly quickly - in $O(n \log n)$ time. We can call SORTING an *easy* problem.
- There are potentially 'hard' problems out there. How do we know which is which?

Easy vs hard problems

- We saw in previous lectures that SORTING can be solved fairly quickly - in $O(n \log n)$ time. We can call SORTING an *easy* problem.
- There are potentially 'hard' problems out there. How do we know which is which?
- Reduction is a powerful tool to accomplish this.

Interval scheduling: the decision problem

Input: a collection of intervals

Interval scheduling: the decision problem

Input: a collection of intervals

$$C = ([a_0, b_0], \dots, [a_{n-1}, b_{n-1}]), \quad a_i, b_i \in \mathbb{R}, \quad a_i \leq b_i.$$

Output: YES if

Interval scheduling: the decision problem

Input: a collection of intervals

$$C = ([a_0, b_0], \dots, [a_{n-1}, b_{n-1}]), \quad a_i, b_i \in \mathbb{R}, \quad a_i \leq b_i.$$

Output: YES if

$$[a_i, b_i] \cap [a_j, b_j] = \emptyset \quad \text{for every } i \neq j.$$

NO otherwise.

Interval scheduling: the decision problem

Input: a collection of intervals

$$C = ([a_0, b_0], \dots, [a_{n-1}, b_{n-1}]), \quad a_i, b_i \in \mathbb{R}, \quad a_i \leq b_i.$$

Output: YES if

$$[a_i, b_i] \cap [a_j, b_j] = \emptyset \quad \text{for every } i \neq j.$$

NO otherwise.

Q: What are the answers for $\{[1, 3], [3, 5]\}$ and $\{[1, 3], [4, 5]\}$?

Interval scheduling: the decision problem

Input: a collection of intervals

$$C = ([a_0, b_0], \dots, [a_{n-1}, b_{n-1}]), \quad a_i, b_i \in \mathbb{R}, \quad a_i \leq b_i.$$

Output: YES if

$$[a_i, b_i] \cap [a_j, b_j] = \emptyset \quad \text{for every } i \neq j.$$

NO otherwise.

Q: What are the answers for $\{[1, 3], [3, 5]\}$ and $\{[1, 3], [4, 5]\}$?

The first is NO; the second is YES.

Checking every pair

Q: How can we solve INTERVALSCHEDULING-DECISION in $O(n^2)$ time?

Checking every pair

Q: How can we solve INTERVALSCHEDULING-DECISION in $O(n^2)$ time?

- Check each pair of intervals. If any pair overlaps, return NO.
- If no pair overlaps, return YES.

Checking every pair

Q: How can we solve INTERVALSCHEDULING-DECISION in $O(n^2)$ time?

- Check each pair of intervals. If any pair overlaps, return NO.
- If no pair overlaps, return YES.

There are $\binom{n}{2}$ pairs and each check takes $O(1)$ time.

Total time: $O(n^2)$. Is there a faster algorithm?

Interval scheduling via sorting

Q: How should we choose the keys and values?

Interval scheduling via sorting

Q: How should we choose the keys and values?

1. Given interval $[a_i, b_i]$: start time a_i is the key, end time b_i is the value.

Interval scheduling via sorting

Q: How should we choose the keys and values?

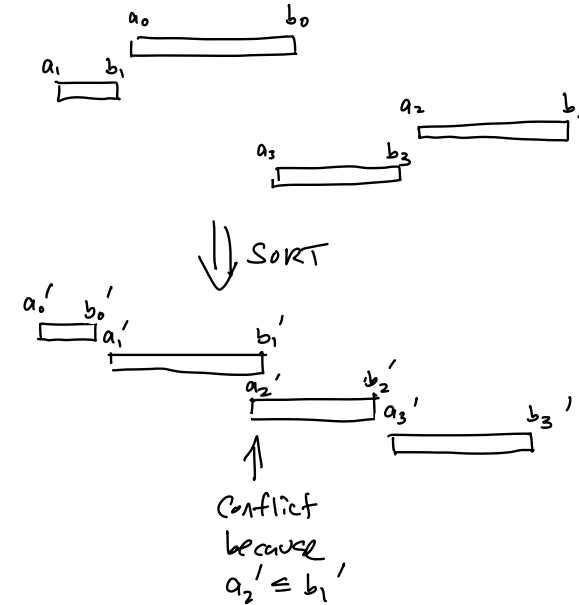
1. Given interval $[a_i, b_i]$: start time a_i is the key, end time b_i is the value.
2. Sort the key-value pairs by start time.

Interval scheduling via sorting

Q: How should we choose the keys and values?

1. Given interval $[a_i, b_i]$: start time a_i is the key, end time b_i is the value.
2. Sort the key-value pairs by start time.
3. Check if consecutive intervals (in the sorted order) overlap.

If no overlap, return YES.



Interval scheduling via sorting

Algorithm 1: IntSchedule(C)

Input: Intervals $C = ([a_0, b_0], \dots, [a_{n-1}, b_{n-1}])$.

Output: YES if the intervals are pairwise disjoint; NO otherwise.

```
1 Set  $A =$  _____  
2 Let  $A' = ((a'_0, b'_0), \dots, (a'_{n-1}, b'_{n-1})) =$  _____  
3 foreach  $i = 1, \dots, n - 1$  do  
4   | _____  
5 return YES
```

Q: What key-value pairs are in A ?

Interval scheduling via sorting

Algorithm 1: IntSchedule(C)

Input: Intervals $C = ([a_0, b_0], \dots, [a_{n-1}, b_{n-1}])$.

Output: YES if the intervals are pairwise disjoint; NO otherwise.

```
1 Set  $A = ((a_0, b_0), \dots, (a_{n-1}, b_{n-1}))$ 
2 Let  $A' = ((a'_0, b'_0), \dots, (a'_{n-1}, b'_{n-1})) = \underline{\hspace{2cm}}$ 
3 foreach  $i = 1, \dots, n - 1$  do
4    $\underline{\hspace{2cm}}$ 
5 return YES
```

Q: Which algorithm produces A' ?

Interval scheduling via sorting

Algorithm 1: IntSchedule(C)

Input: Intervals $C = ([a_0, b_0], \dots, [a_{n-1}, b_{n-1}])$.

Output: YES if the intervals are pairwise disjoint; NO otherwise.

```
1 Set  $A = ((a_0, b_0), \dots, (a_{n-1}, b_{n-1}))$ 
2 Let  $A' = ((a'_0, b'_0), \dots, (a'_{n-1}, b'_{n-1})) = \text{MergeSort}(A)$ 
3 foreach  $i = 1, \dots, n - 1$  do
4   | _____
5 return YES
```

Q: What condition detects a conflict?

Interval scheduling via sorting

Algorithm 1: IntSchedule(C)

Input: Intervals $C = ([a_0, b_0], \dots, [a_{n-1}, b_{n-1}])$.

Output: YES if the intervals are pairwise disjoint; NO otherwise.

- 1 Set $A = ((a_0, b_0), \dots, (a_{n-1}, b_{n-1}))$
 - 2 Let $A' = ((a'_0, b'_0), \dots, (a'_{n-1}, b'_{n-1})) = \text{MergeSort}(A)$
 - 3 **foreach** $i = 1, \dots, n - 1$ **do**
 - 4 **if** $a'_i \leq b'_{i-1}$ **then return** NO
 - 5 **return** YES
-

Q: What does this return when $n = 1$?

Interval scheduling via sorting

Algorithm 1: IntSchedule(C)

Input: Intervals $C = ([a_0, b_0], \dots, [a_{n-1}, b_{n-1}])$.

Output: YES if the intervals are pairwise disjoint; NO otherwise.

- 1 Set $A = ((a_0, b_0), \dots, (a_{n-1}, b_{n-1}))$
 - 2 Let $A' = ((a'_0, b'_0), \dots, (a'_{n-1}, b'_{n-1})) = \text{MergeSort}(A)$
 - 3 **foreach** $i = 1, \dots, n - 1$ **do**
 - 4 **if** $a'_i \leq b'_{i-1}$ **then return** NO
 - 5 **return** YES
-

It returns YES: the loop makes no comparisons.

IntSchedule: running time

Q: What does each phase cost?

- Form the array of key–value pairs:

IntSchedule: running time

Q: What does each phase cost?

- Form the array of key–value pairs: $O(n)$.
- Sort using MergeSort:

IntSchedule: running time

Q: What does each phase cost?

- Form the array of key–value pairs: $O(n)$.
- Sort using MergeSort: $O(n \log n)$.
- Check adjacent intervals:

IntSchedule: running time

Q: What does each phase cost?

- Form the array of key–value pairs: $O(n)$.
- Sort using MergeSort: $O(n \log n)$.
- Check adjacent intervals: $O(n)$.

IntSchedule: running time

Q: What does each phase cost?

- Form the array of key–value pairs: $O(n)$.
- Sort using MergeSort: $O(n \log n)$.
- Check adjacent intervals: $O(n)$.

Total time: $O(n) + O(n \log n) + O(n) = O(n \log n)$.

IntSchedule: correctness

Q: Why is checking adjacent intervals sufficient? **If the intervals are disjoint, the algorithm returns YES.**

IntSchedule: correctness

Q: Why is checking adjacent intervals sufficient? **If the intervals are disjoint, the algorithm returns YES.**

This can be argued inspecting Lines 3,4.

If the algorithm returns YES, the intervals are disjoint.

IntSchedule: correctness

Q: Why is checking adjacent intervals sufficient? **If the intervals are disjoint, the algorithm returns YES.**

This can be argued inspecting Lines 3,4.

If the algorithm returns YES, the intervals are disjoint.

Every adjacent test passed (Lines 3,4), so

$$a'_0 \leq b'_0 < a'_1 \leq b'_1 < \cdots < a'_{n-1} \leq b'_{n-1}.$$

Thus every interval ends before any later interval starts.

IntSchedule: correctness

Q: Why is checking adjacent intervals sufficient? **If the intervals are disjoint, the algorithm returns YES.**

This can be argued inspecting Lines 3,4.

If the algorithm returns YES, the intervals are disjoint.

Every adjacent test passed (Lines 3,4), so

$$a'_0 \leq b'_0 < a'_1 \leq b'_1 < \cdots < a'_{n-1} \leq b'_{n-1}.$$

Thus every interval ends before any later interval starts.

Q: Would this correctness proof change, if MergeSort was replaced by InsertionSort or Exhaustive-Search Sort?

Interval scheduling on a finite universe

Suppose all endpoints a_i, b_i lie in $[U] = \{0, \dots, U - 1\}$. **Q:** Can we solve the problem in $O(n + U)$ time?

Interval scheduling on a finite universe

Suppose all endpoints a_i, b_i lie in $[U] = \{0, \dots, U - 1\}$. **Q:** Can we solve the problem in $O(n + U)$ time?

- Keep the key-value pairs (a_i, b_i) and the adjacent-interval test.
- Replace MergeSort with SingletonBucketSort on universe $[U]$.

$$O(n) + O(n + U) + O(n) = O(n + U).$$

Oracle

Let $\Gamma = (\mathcal{J}, \mathcal{P}, g)$ be a computational problem.

Oracle for Γ

On a query $y \in \mathcal{J}$, it returns an element of $g(y)$, or $\perp$ if $g(y) = \emptyset$.

A hypothetical subroutine that always gives a valid answer (if exists).

Oracle

Let $\Gamma = (\mathcal{J}, \mathcal{P}, g)$ be a computational problem.

Oracle for Γ

On a query $y \in \mathcal{J}$, it returns an element of $g(y)$, or $\perp$ if $g(y) = \emptyset$.

A hypothetical subroutine that always gives a valid answer (if exists).

Example usage:

Algorithm 2: Something that solves Int Scheduling(C)

Input: Intervals $C = ([a_0, b_0], \dots, [a_{n-1}, b_{n-1}])$.

Output: YES if the intervals are pairwise disjoint; NO otherwise.

- 1 Set $A = ((a_0, b_0), \dots, (a_{n-1}, b_{n-1}))$
 - 2 Let $A' = ((a'_0, b'_0), \dots, (a'_{n-1}, b'_{n-1})) = \text{SORTINGOracle}(A)$
 - 3 **foreach** $i = 1, \dots, n - 1$ **do**
 - 4 **if** $a'_i \leq b'_{i-1}$ **then return** NO
 - 5 **return** YES
-

Reductions

Let $\Pi = (\mathcal{I}, \mathcal{O}, f)$ and $\Gamma = (\mathcal{J}, \mathcal{P}, g)$ be computational problems.

Reductions

Let $\Pi = (\mathcal{I}, \mathcal{O}, f)$ and $\Gamma = (\mathcal{J}, \mathcal{P}, g)$ be computational problems.

Reduction

An algorithm that solves Π using as a subroutine *any oracle* that solves Γ .

Reductions

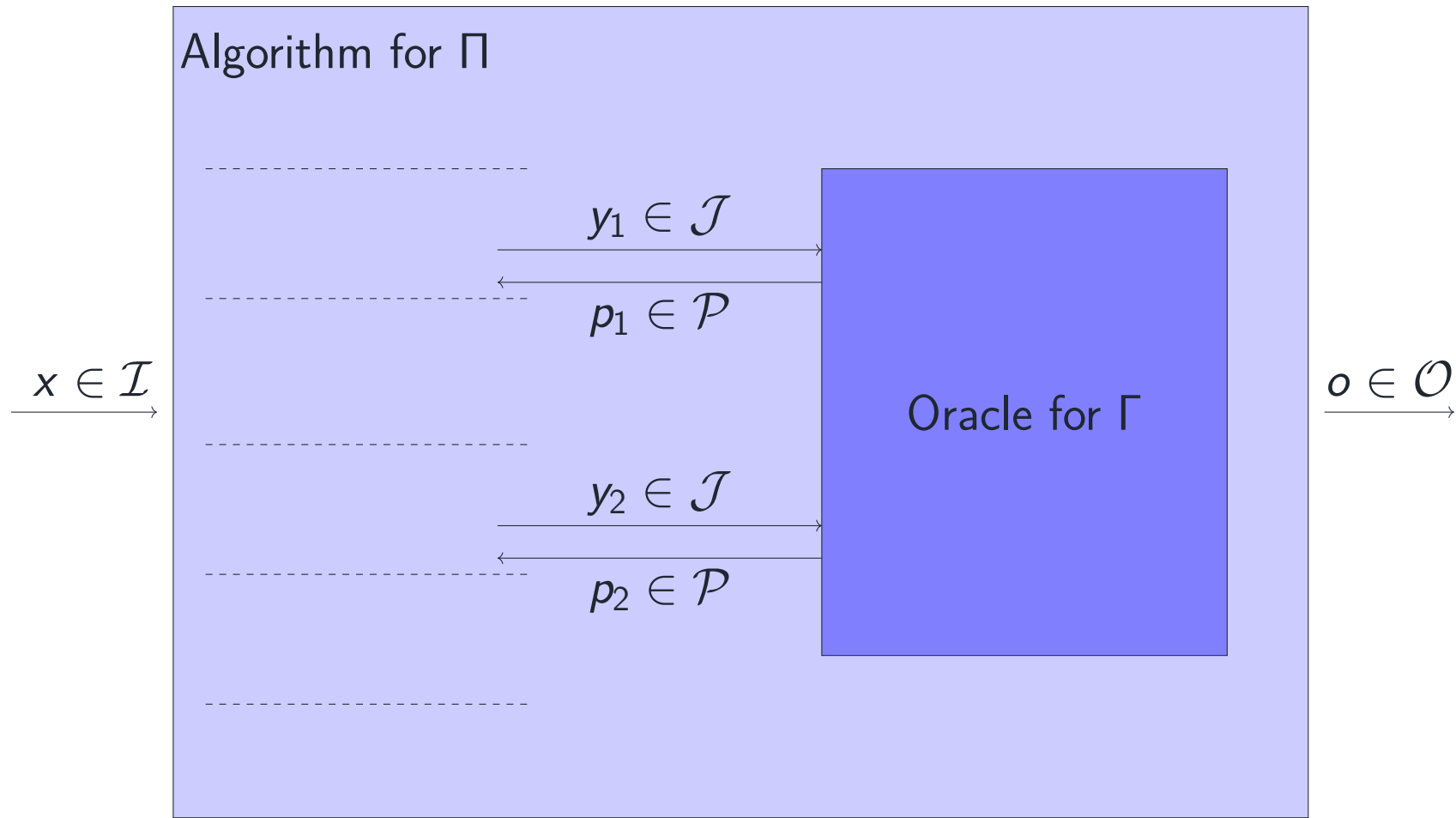
Let $\Pi = (\mathcal{I}, \mathcal{O}, f)$ and $\Gamma = (\mathcal{J}, \mathcal{P}, g)$ be computational problems.

Reduction

An algorithm that solves Π using as a subroutine *any oracle* that solves Γ .

For the previous example: **Something that solves Int Scheduling** is really a **Reduction** from IntSchedule — Decision to SORTING.

Visualizing reduction



Notation and the cost of a reduction

- $\Pi \leq \Gamma$: there is a reduction from Π to Γ .

Notation and the cost of a reduction

- $\Pi \leq \Gamma$: there is a reduction from Π to Γ . Read: “ Π reduces to Γ .”

Notation and the cost of a reduction

- $\Pi \leq \Gamma$: there is a reduction from Π to Γ . Read: “ Π reduces to Γ .”
- $\Pi \leq_{T,h} \Gamma$: on inputs of size n , the reduction takes $O(T(n))$ time and makes one oracle call of size at most $h(n)$.

Notation and the cost of a reduction

- $\Pi \leq \Gamma$: there is a reduction from Π to Γ . Read: “ Π reduces to Γ .”
- $\Pi \leq_{T,h} \Gamma$: on inputs of size n , the reduction takes $O(T(n))$ time and makes one oracle call of size at most $h(n)$.
- $\Pi \leq_{T,q \otimes h} \Gamma$: the reduction takes $O(T(n))$ time and makes at most $q(n)$ calls, each of size at most $h(n)$.

Convention: each oracle call counts as one time step.

The interval scheduling reduction

Q: Fill in the blanks

INTERVALSCHEDULING-DECISION $\leq$ _____ , _____ SORTING.

The interval scheduling reduction

Q: Fill in the blanks

INTERVALSCHEDULING-DECISION $\leq$ _____ , _____ SORTING.

INTERVALSCHEDULING-DECISION $\leq_{O(n), n}$ SORTING.

- Build the array and check adjacent key-value pairs: $O(n)$ steps.
- Sorting oracle called on array of size n .

The reduction takes $O(n)$ time. Substituting MergeSort costs $O(n \log n)$ overall.

What reductions imply: solvability

(Lemma 4.5 in HV)

Assume $\Pi \leq \Gamma$.

1. If an algorithm solves Γ , then **an algorithm solves Π** .

What reductions imply: solvability

(Lemma 4.5 in HV)

Assume $\Pi \leq \Gamma$.

1. If an algorithm solves Γ , then **an algorithm solves Π** . *Proof:* replace each oracle call in the reduction with the algorithm for Γ .
2. If no algorithm solves Π , then **no algorithm solves Γ** .
Proof: this is the contrapositive of the first statement.

What reductions imply: running time

(Lemma 4.5 of HV)

Assume $\Pi \leq_{T_R, q \otimes h} \Gamma$, and an algorithm for Γ has worst-case running time $T_A(m)$ on inputs of size at most m . **Q:** What running time do we obtain for Π ?

What reductions imply: running time

(Lemma 4.5 of HV)

Assume $\Pi \leq_{T_R, q \otimes h} \Gamma$, and an algorithm for Γ has worst-case running time $T_A(m)$ on inputs of size at most m . **Q:** What running time do we obtain for Π ?

$$\underbrace{O(T_R(n))}_{\text{reduction runtime}} + \underbrace{O(q(n) T_A(h(n)))}_{\text{replace every oracle call}}.$$

Proof: run the reduction and substitute the algorithm for each call. There are at most $q(n)$ calls, each on an input of size at most $h(n)$.

Q: For interval scheduling via sorting, what are T_R , q , h , and T_A ?

What reductions imply: running time

(Lemma 4.5 of HV)

Assume $\Pi \leq_{T_R, q \otimes h} \Gamma$, and an algorithm for Γ has worst-case running time $T_A(m)$ on inputs of size at most m . **Q:** What running time do we obtain for Π ?

$$\underbrace{O(T_R(n))}_{\text{reduction runtime}} + \underbrace{O(q(n) T_A(h(n)))}_{\text{replace every oracle call}}.$$

Proof: run the reduction and substitute the algorithm for each call. There are at most $q(n)$ calls, each on an input of size at most $h(n)$.

Q: For interval scheduling via sorting, what are T_R , q , h , and T_A ?

$$T_R(n) = n, \quad q(n) = 1, \quad h(n) = n, \quad T_A(n) = O(n \log n).$$

What reductions imply: running time

Assume $\Pi \leq_{T_R, q \otimes h} \Gamma$. If Π has no algorithm with running time $O(T_R(n) + q(n)T_A(h(n)))$, then ...

What reductions imply: running time

Assume $\Pi \leq_{T_R, q \otimes h} \Gamma$. If Π has no algorithm with running time $O(T_R(n) + q(n)T_A(h(n)))$, then ...

Γ has no algorithm with running time $O(T_A(n))$.

This is the contrapositive of the runtime statement.

Q: Does $\Pi \leq \Gamma$ also mean that a fast algorithm for Π gives a fast algorithm for Γ ?

What reductions imply: running time

Assume $\Pi \leq_{T_R, q \otimes h} \Gamma$. If Π has no algorithm with running time $O(T_R(n) + q(n)T_A(h(n)))$, then ...

Γ has no algorithm with running time $O(T_A(n))$.

This is the contrapositive of the runtime statement.

Q: Does $\Pi \leq \Gamma$ also mean that a fast algorithm for Π gives a fast algorithm for Γ ?

No. An easy problem can reduce to a hard problem: solve the easy problem directly and ignore the oracle's answer.

What reductions imply: running time

Assume $\Pi \leq_{T_R, q \otimes h} \Gamma$. If Π has no algorithm with running time $O(T_R(n) + q(n)T_A(h(n)))$, then ...

Γ has no algorithm with running time $O(T_A(n))$.

This is the contrapositive of the runtime statement.

Q: Does $\Pi \leq \Gamma$ also mean that a fast algorithm for Π gives a fast algorithm for Γ ?

No. An easy problem can reduce to a hard problem: solve the easy problem directly and ignore the oracle's answer.

Rare example of reducing an easy problem to a hard problem: How to install a light bulb? By reduction to 'rotating the universe'.

A new request for airtime

We have verified that the original intervals are disjoint. An interval arrives: $[a^*, b^*]$.

A new request for airtime

We have verified that the original intervals are disjoint. An interval arrives: $[a^*, b^*]$.

Q: Must we spend $O(n \log n)$ time again to test for a conflict?

A new request for airtime

We have verified that the original intervals are disjoint. An interval arrives: $[a^*, b^*]$.

Q: Must we spend $O(n \log n)$ time again to test for a conflict?

- Keep the intervals in the sorted array from IntSchedule.

A new request for airtime

We have verified that the original intervals are disjoint. An interval arrives: $[a^*, b^*]$.

Q: Must we spend $O(n \log n)$ time again to test for a conflict?

- Keep the intervals in the sorted array from IntSchedule.
- Binary search for the last start time at most a^* .

A new request for airtime

We have verified that the original intervals are disjoint. An interval arrives: $[a^*, b^*]$.

Q: Must we spend $O(n \log n)$ time again to test for a conflict?

- Keep the intervals in the sorted array from IntSchedule.
- Binary search for the last start time at most a^* .
- Check the preceding interval's end and the following interval's start.

Preprocess once; answer each new 'query' in $O(\log n)$ time.

Each query tests a possible new interval. The stored schedule stays fixed.

Static abstract data types

A *static abstract data type* is a quadruple

$$\Pi = (\mathcal{I}, \mathcal{O}, \mathcal{Q}, f),$$

where:

- $\mathcal{I}$ is the set of inputs x , and $\mathcal{O}$ is the set of outputs.
- $\mathcal{Q}$ is

Static abstract data types

A *static abstract data type* is a quadruple

$$\Pi = (\mathcal{I}, \mathcal{O}, \mathcal{Q}, f),$$

where:

- $\mathcal{I}$ is the set of inputs x , and $\mathcal{O}$ is the set of outputs.
- $\mathcal{Q}$ is the set of possible *queries*.

Static abstract data types

A *static abstract data type* is a quadruple

$$\Pi = (\mathcal{I}, \mathcal{O}, \mathcal{Q}, f),$$

where:

- $\mathcal{I}$ is the set of inputs x , and $\mathcal{O}$ is the set of outputs.
- $\mathcal{Q}$ is the set of possible *queries*.
- For every $x \in \mathcal{I}$ and $q \in \mathcal{Q}$,

Static abstract data types

A *static abstract data type* is a quadruple

$$\Pi = (\mathcal{I}, \mathcal{O}, \mathcal{Q}, f),$$

where:

- $\mathcal{I}$ is the set of inputs x , and $\mathcal{O}$ is the set of outputs.
- $\mathcal{Q}$ is the set of possible *queries*.
- For every $x \in \mathcal{I}$ and $q \in \mathcal{Q}$,
 $f(x, q) \subseteq \mathcal{O}$ is the set of *valid answers*.

The input is known first. Queries arrive later.

Type: Static predecessors and successors

Input: $x = ((K_0, V_0), \dots, (K_{n-1}, V_{n-1}))$, with keys $\in \mathbb{R}$.

Queries: return a key–value pair satisfying the requested condition.

- $\text{search}(K)$:

Type: Static predecessors and successors

Input: $x = ((K_0, V_0), \dots, (K_{n-1}, V_{n-1}))$, with keys $\in \mathbb{R}$.

Queries: return a key–value pair satisfying the requested condition.

- $\text{search}(K)$: return some (K_i, V_i) with $K_i = K$.
- $\text{predecessor}(K)$:

Type: Static predecessors and successors

Input: $x = ((K_0, V_0), \dots, (K_{n-1}, V_{n-1}))$, with keys $\in \mathbb{R}$.

Queries: return a key–value pair satisfying the requested condition.

- $\text{search}(K)$: return some (K_i, V_i) with $K_i = K$.
- $\text{predecessor}(K)$: return some (K_i, V_i) with $K_i = \max\{K_j : K_j < K\}$.
- $\text{successor}(K)$:

Type: Static predecessors and successors

Input: $x = ((K_0, V_0), \dots, (K_{n-1}, V_{n-1}))$, with keys $\in \mathbb{R}$.

Queries: return a key–value pair satisfying the requested condition.

- $\text{search}(K)$: return some (K_i, V_i) with $K_i = K$.
- $\text{predecessor}(K)$: return some (K_i, V_i) with $K_i = \max\{K_j : K_j < K\}$.
- $\text{successor}(K)$: return some (K_i, V_i) with $K_i = \min\{K_j : K_j > K\}$.

Q: Why might a query have no valid answer? More than one valid answer?

Example

$$x = ((2, a), (5, b), (5, c), (9, d)).$$

Q: List all valid answers to each query.

- `search(5)`:

Example

$$x = ((2, a), (5, b), (5, c), (9, d)).$$

Q: List all valid answers to each query.

- `search(5)`: $(5, b)$ or $(5, c)$.
- `predecessor(5)`:

Example

$$x = ((2, a), (5, b), (5, c), (9, d)).$$

Q: List all valid answers to each query.

- `search(5)`: $(5, b)$ or $(5, c)$.
- `predecessor(5)`: $(2, a)$.
- `successor(5)`:

Example

$$x = ((2, a), (5, b), (5, c), (9, d)).$$

Q: List all valid answers to each query.

- $\text{search}(5)$: $(5, b)$ or $(5, c)$.
- $\text{predecessor}(5)$: $(2, a)$.
- $\text{successor}(5)$: $(9, d)$.
- $\text{predecessor}(2)$ and $\text{search}(7)$:

Example

$$x = ((2, a), (5, b), (5, c), (9, d)).$$

Q: List all valid answers to each query.

- $\text{search}(5)$: $(5, b)$ or $(5, c)$.
- $\text{predecessor}(5)$: $(2, a)$.
- $\text{successor}(5)$: $(9, d)$.
- $\text{predecessor}(2)$ and $\text{search}(7)$: neither has a valid answer; return $\perp$.

What is a static data structure?

A *static data structure* for $\Pi = (\mathcal{I}, \mathcal{O}, \mathcal{Q}, f)$ is a pair of algorithms (Preprocess, Eval).

What is a static data structure?

A *static data structure* for $\Pi = (\mathcal{I}, \mathcal{O}, \mathcal{Q}, f)$ is a pair of algorithms (Preprocess, Eval).

- If $f(x, q) \neq \emptyset$, then

$$\text{Eval}(\text{Preprocess}(x), q) \in f(x, q).$$

What is a static data structure?

A *static data structure* for $\Pi = (\mathcal{I}, \mathcal{O}, \mathcal{Q}, f)$ is a pair of algorithms (Preprocess, Eval).

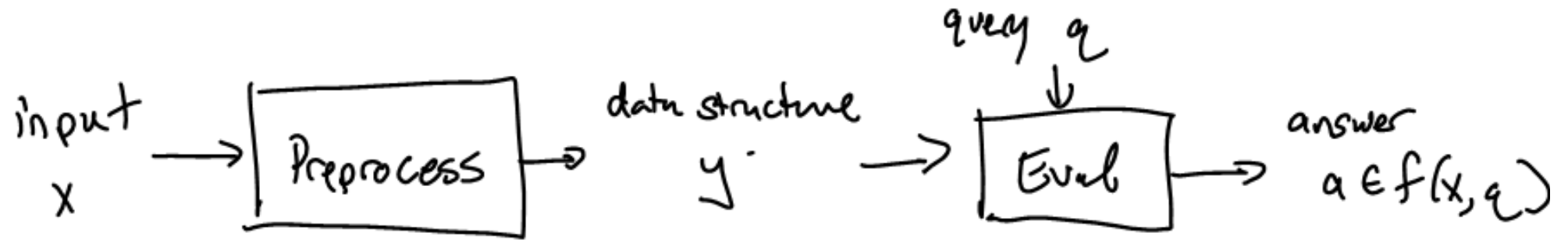
- If $f(x, q) \neq \emptyset$, then

$$\text{Eval}(\text{Preprocess}(x), q) \in f(x, q).$$

- If $f(x, q) = \emptyset$, then

$$\text{Eval}(\text{Preprocess}(x), q) = \perp, \quad \perp \notin \mathcal{O}.$$

Preprocessing and evaluation



The stored object $y = \text{Preprocess}(x)$ is an *encoding* of the input.

- Main goal: fast query evaluation.
- Also measure preprocessing time and the space used by the encoding.

Revisiting sorted array

Theorem. The abstract data type ‘Static predecessors and successors’ has a data structure with:

- Query time for search, predecessor, and successor:

Revisiting sorted array

Theorem. The abstract data type ‘Static predecessors and successors’ has a data structure with:

- Query time for search, predecessor, and successor: $O(\log n)$.
- Preprocessing time:

Revisiting sorted array

Theorem. The abstract data type ‘Static predecessors and successors’ has a data structure with:

- Query time for search, predecessor, and successor: $O(\log n)$.
- Preprocessing time: $O(n \log n)$.
- Space for the encoding:

Revisiting sorted array

Theorem. The abstract data type ‘Static predecessors and successors’ has a data structure with:

- Query time for search, predecessor, and successor: $O(\log n)$.
- Preprocessing time: $O(n \log n)$.
- Space for the encoding: $O(n)$.

Q: What are the two algorithms?

Revisiting sorted array

Theorem. The abstract data type ‘Static predecessors and successors’ has a data structure with:

- Query time for search, predecessor, and successor: $O(\log n)$.
- Preprocessing time: $O(n \log n)$.
- Space for the encoding: $O(n)$.

Q: What are the two algorithms?

Preprocess: sort the key–value pairs. Eval: use binary search.

Other queries on a sorted array

- $\text{min}()$: return a record with the smallest key.
- $\text{rank}(K)$: return the number of records whose keys are strictly less than K .

Q: How long does each query take?

Other queries on a sorted array

- $\text{min}()$: return a record with the smallest key.
- $\text{rank}(K)$: return the number of records whose keys are strictly less than K .

Q: How long does each query take?

- Minimum: return the first record in $O(1)$ time, or $\perp$ if empty.
- Rank: the lower-bound index L counts all records with key $< K$. Find it in $O(\log n)$ time, including when keys repeat.

Other queries on a sorted array

- $\text{min}()$: return a record with the smallest key.
- $\text{rank}(K)$: return the number of records whose keys are strictly less than K .

Q: How long does each query take?

- Minimum: return the first record in $O(1)$ time, or $\perp$ if empty.
- Rank: the lower-bound index L counts all records with key $< K$. Find it in $O(\log n)$ time, including when keys repeat.

Application: range selection. Select records whose keys lie in a range, such as alumni graduating in a chosen decade.

Other queries on a sorted array

- $\text{min}()$: return a record with the smallest key.
- $\text{rank}(K)$: return the number of records whose keys are strictly less than K .

Q: How long does each query take?

- Minimum: return the first record in $O(1)$ time, or $\perp$ if empty.
- Rank: the lower-bound index L counts all records with key $< K$. Find it in $O(\log n)$ time, including when keys repeat.

Application: range selection. Select records whose keys lie in a range, such as alumni graduating in a chosen decade.

Find the first key in the range, then scan until leaving it: $O(\log n + k)$ time to report k records.

Next: dynamic data structures

So far, queries have left the stored dataset unchanged. Next we allow updates:

- $\text{insert}(K, V)$: add a key–value pair.
- $\text{delete}(K)$: remove a stored pair with key K .

Q: In a sorted array, how long can an insertion or deletion take in the worst case?

Next: dynamic data structures

So far, queries have left the stored dataset unchanged. Next we allow updates:

- $\text{insert}(K, V)$: add a key–value pair.
- $\text{delete}(K)$: remove a stored pair with key K .

Q: In a sorted array, how long can an insertion or deletion take in the worst case?

Finding the position takes $O(\log n)$ time, but maintaining sorted order can require shifting $\Theta(n)$ records.

Sorted arrays give fast queries; updates can take $\Theta(n)$ time.